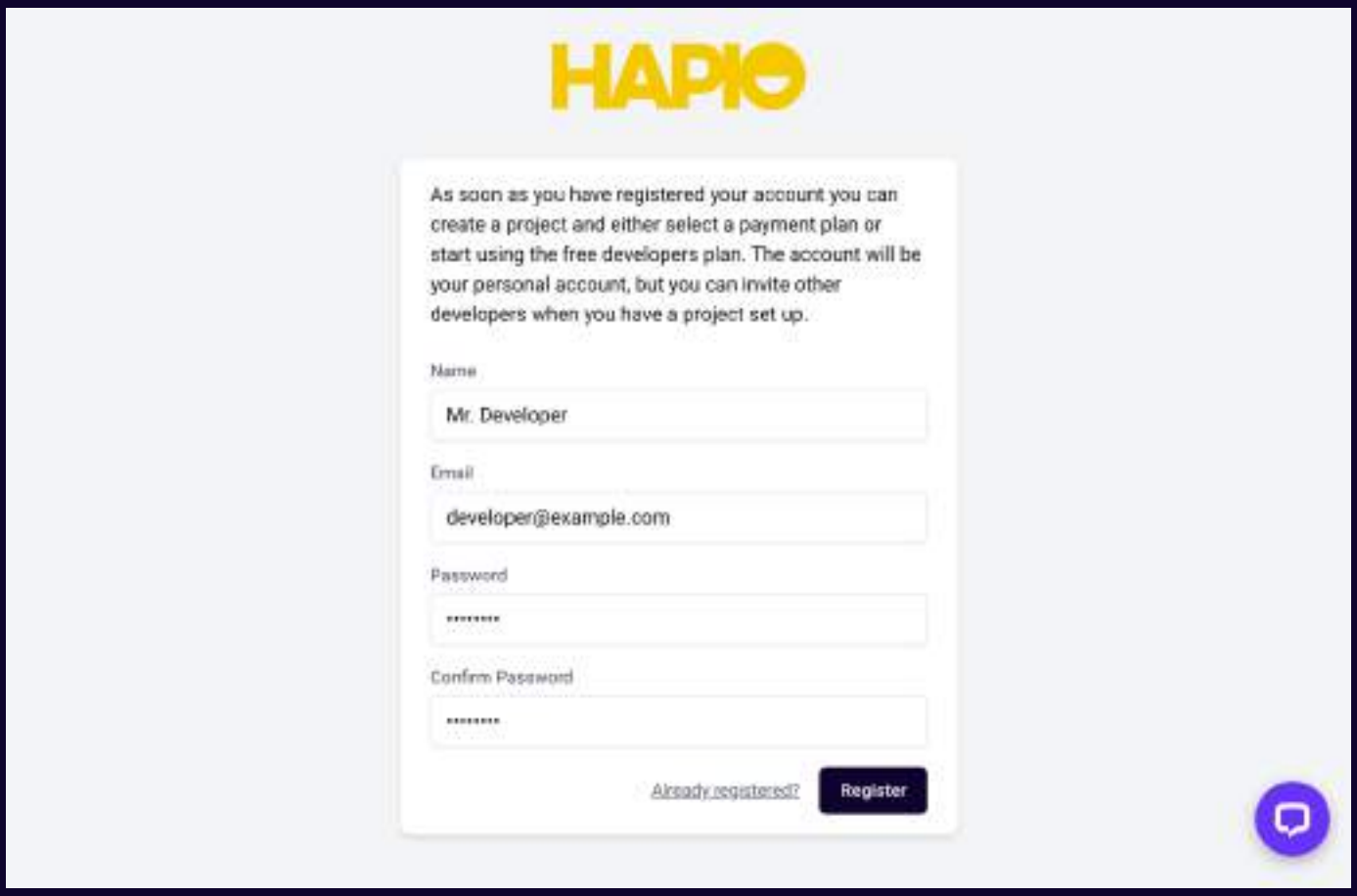
HAPIO

Getting started with Hapio.

This guide will take you through the first steps of getting started with Hapio, and will guide you all the way from registering an account to creating your first booking.

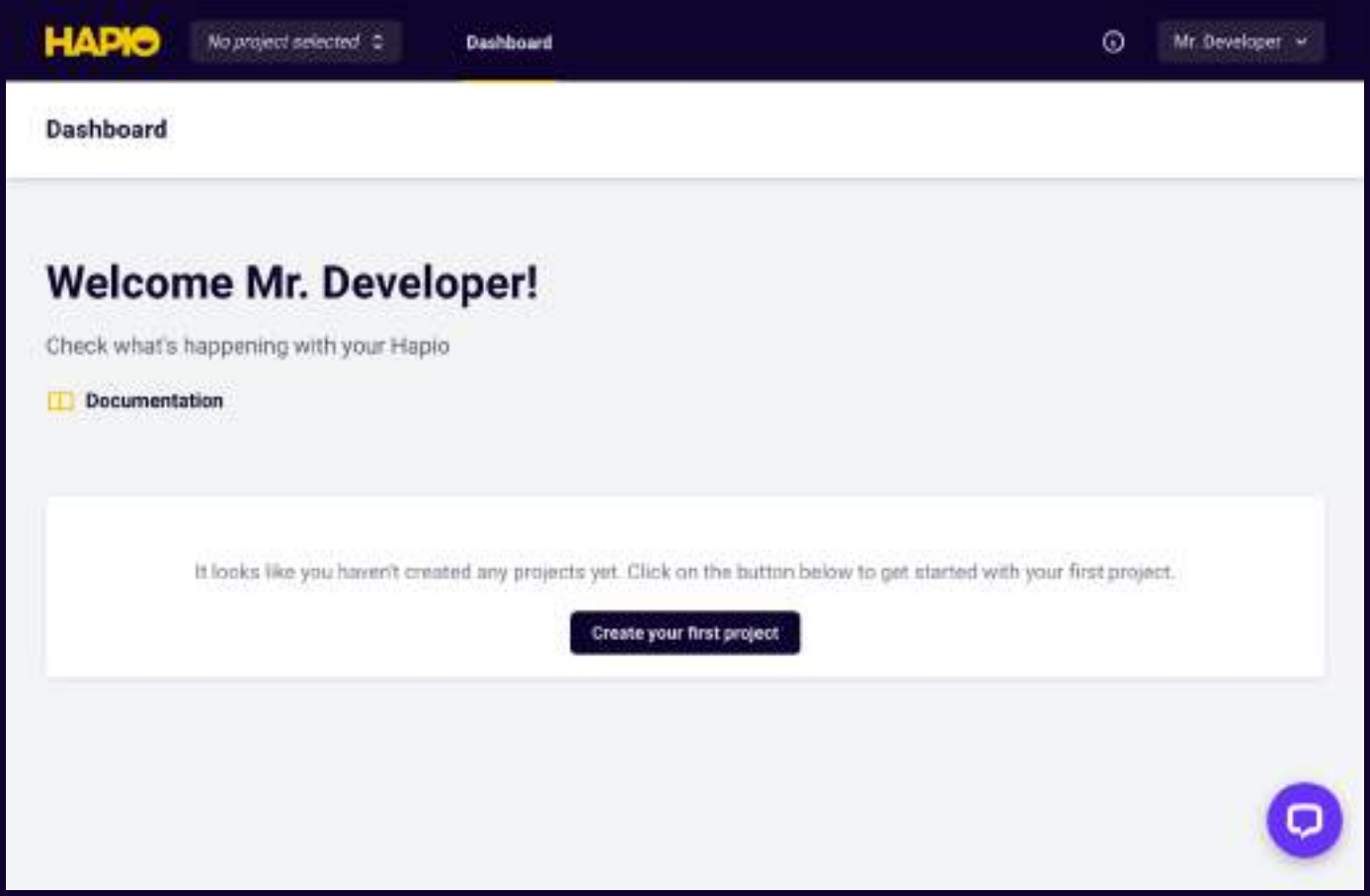
Register

The first step is to register an account in the Hapio Portal. Enter your name, email address, and password, and click on Register. Hapio will then send you an email so that you can verify your account. Remember that multiple people can be invited to join projects, so each person can have their own account in the Hapio Portal.

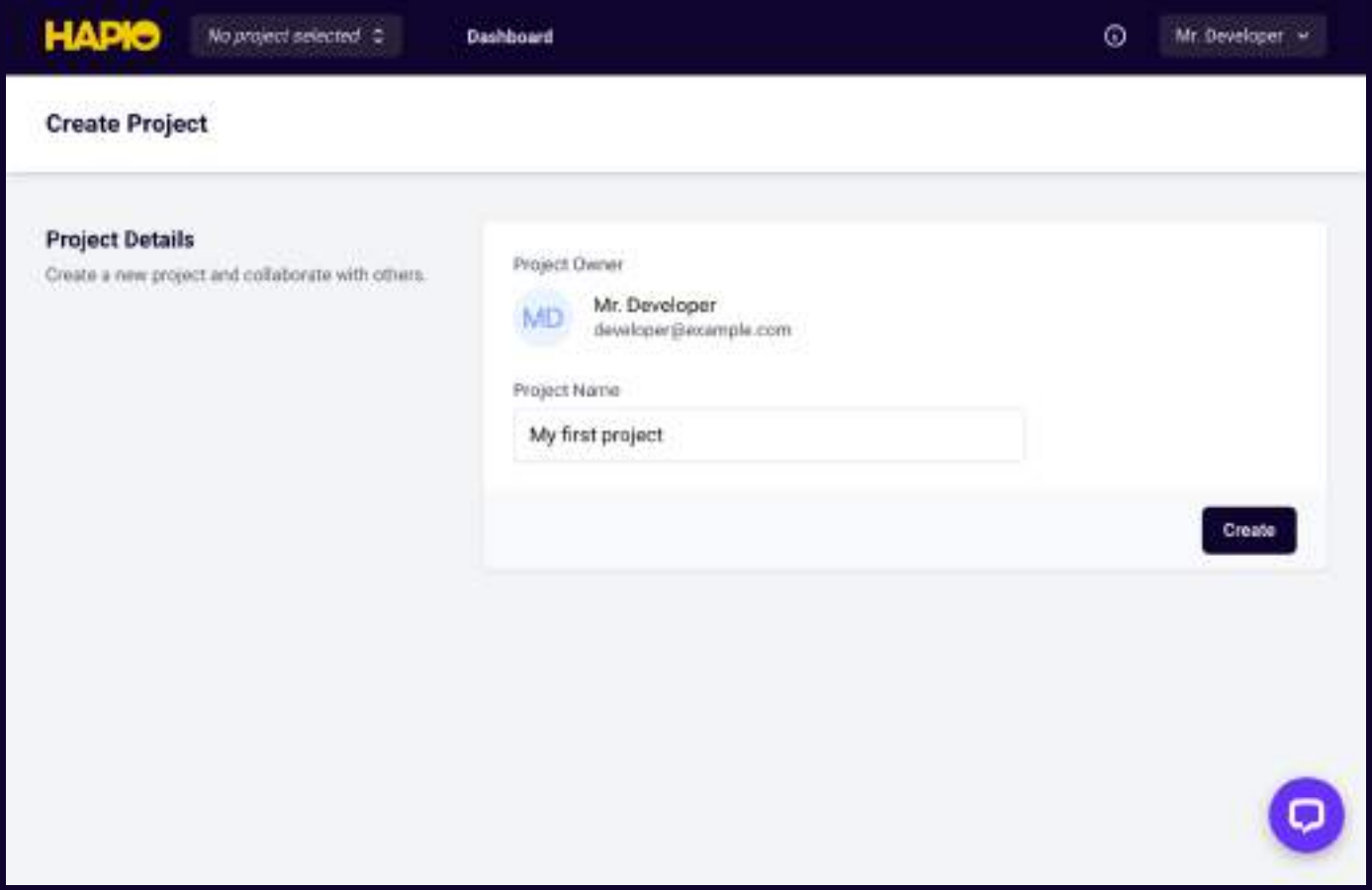


Create your first project

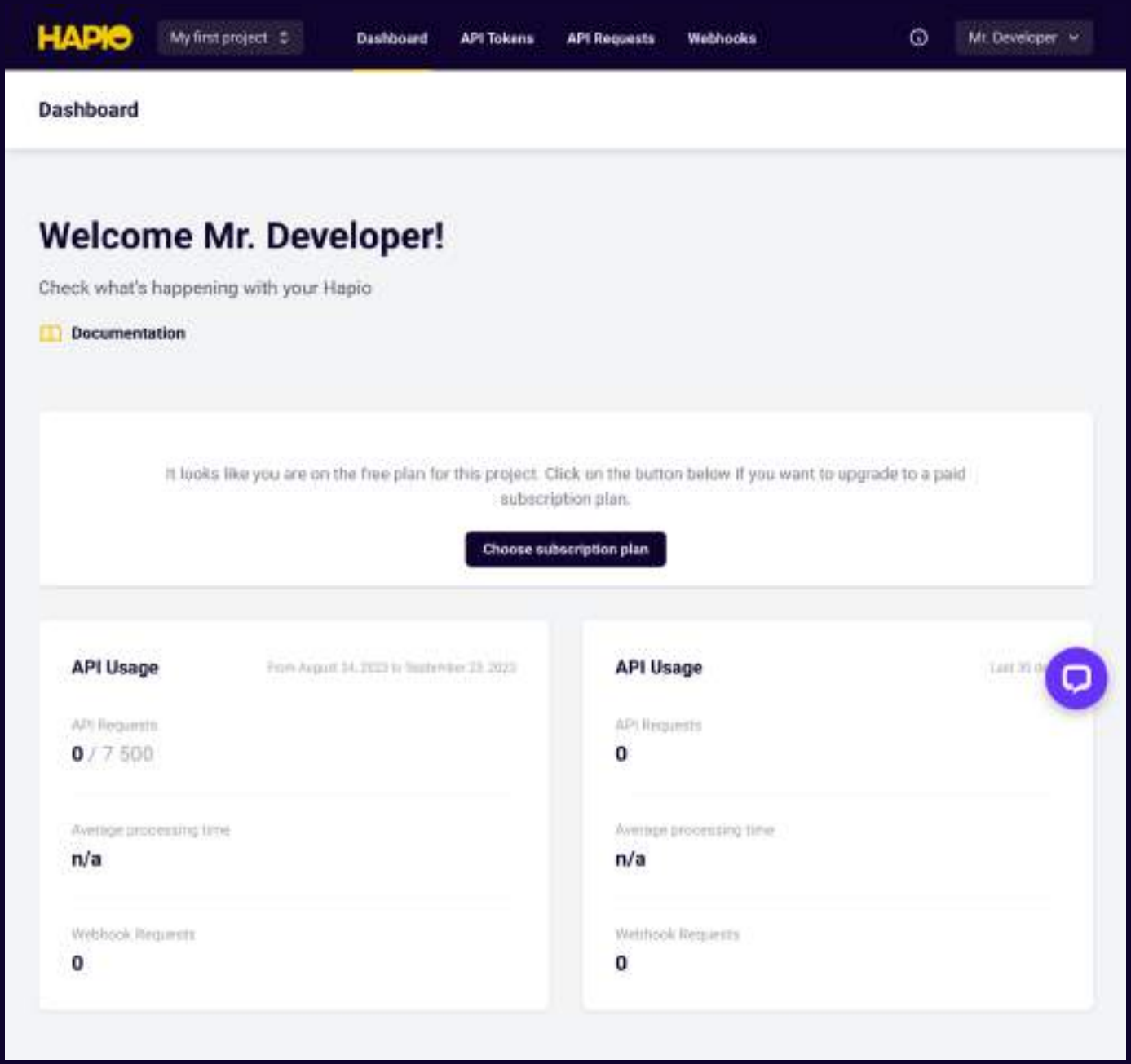
When you have verified your account, you will be logged in to the Hapio Portal. Click on **Create your first project** on the dashboard, or click on the project selector in the top navigation bar, and click on **Create New Project**.



Once you have chosen a name for your project, click on **Create**.

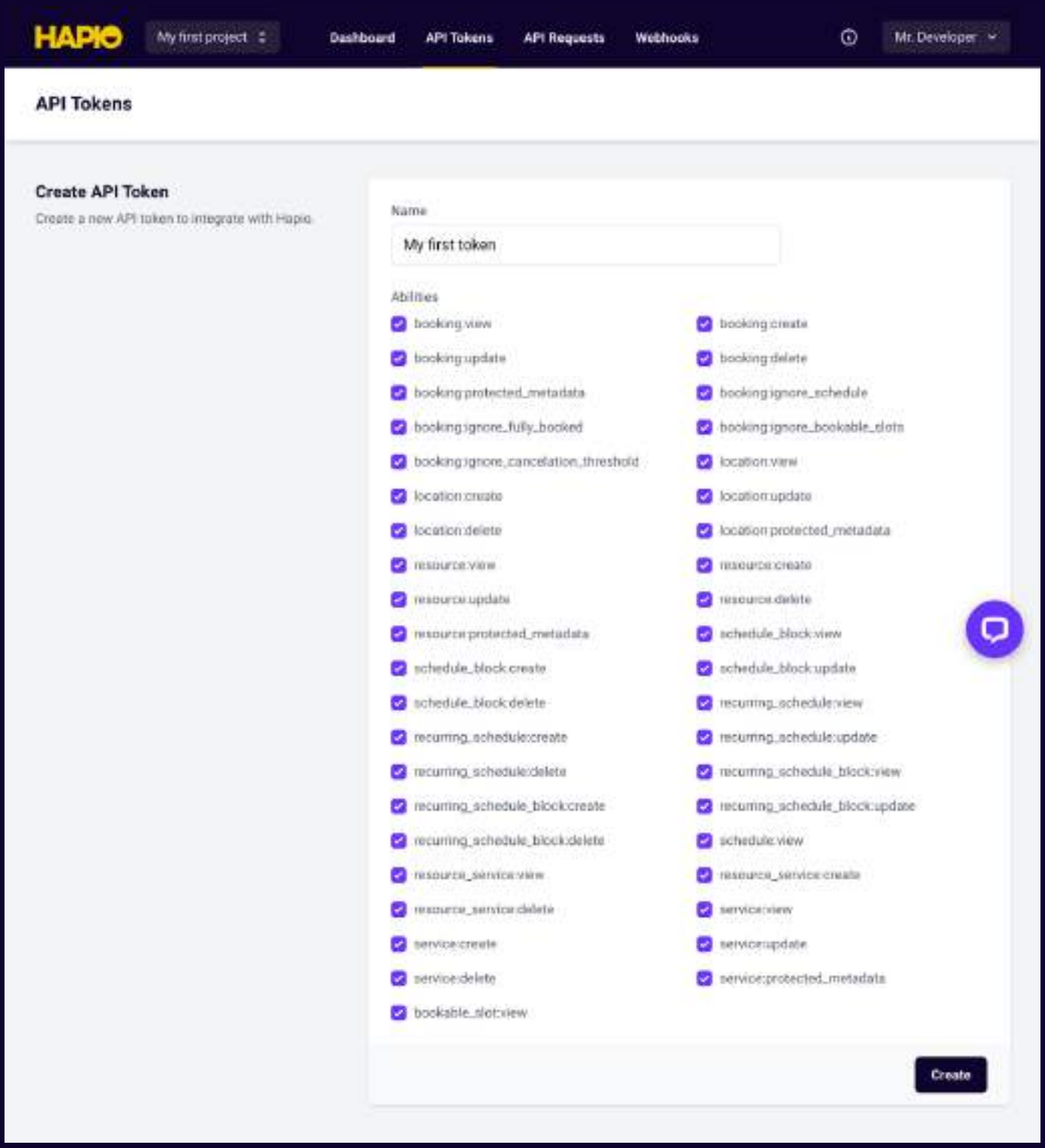


All new projects are on the free plan by default, and this plan is perfect for getting started with Hapio and can be used for development purposes. If you want to increase the limits for your project, you can subscribe to any of the paid plans. Subscription plans are chosen per project, so each project can have a subscription plan that is suitable for that specific project.

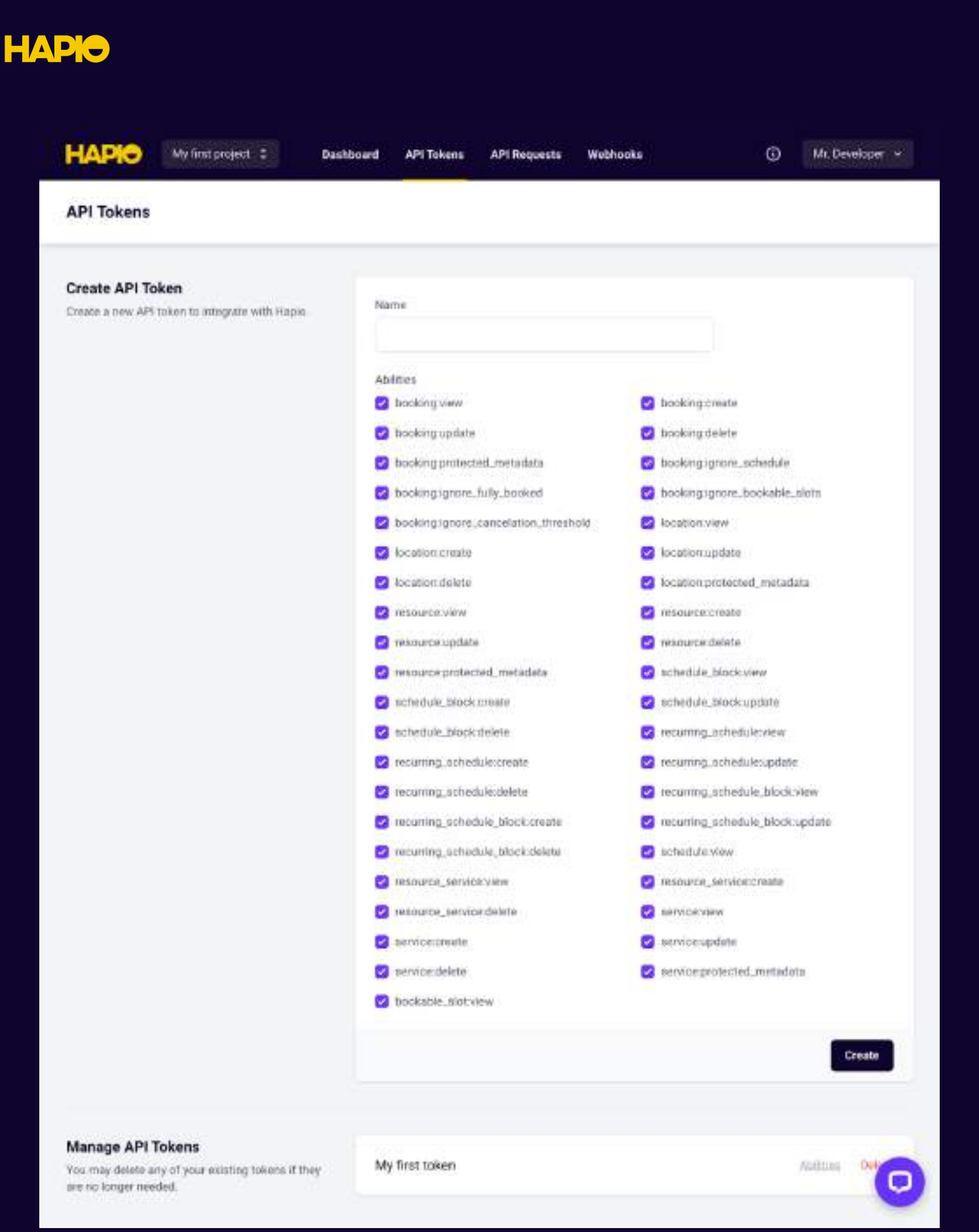


Create an API token

If you look in the project selector in the top navigation bar, your newly created project should be selected as your current project. If it is not selected, click on the project selector and select your project. Now, you can click on **API Tokens** in the top navigation bar, to get to the administration page for your API tokens. This page will list all API tokens that have been created for the project, and will also let you create new tokens. Enter a name for your token, select the abilities that your token should have, and click on **Create**. In this case, we will leave all abilities selected for simplicity, but remember that you should tailor each of your tokens to only have the abilities that it actually needs, in order to increase security.



Once you have created your token, the generated token will be shown for you to copy and store in a secure location. It's important to copy your token now, since this is the only time it will be shown, and it cannot be retrieved at a later point. All created API tokens will be listed on the bottom of the page, and from there you can edit the abilities of each token, and also delete any token if needed.


HAPIO
My first project
Dashboard
API Tokens
API Requests
Webhooks
Mr. Developer
API Tokens
Create API Token
Create a new API token to integrate with Hapio
Name
Abilities
booking:view
booking:create
booking:update
booking:delete
booking:protected_metadata
booking:ignore_schedule
booking:ignore_fully_booked
booking:ignore_bookable_slots
booking:ignore_cancelation_threshold
location:view
location:create
location:update
location:delete
location:protected_metadata
resource:view
resource:create
resource:update
resource:delete
resource:protected_metadata
schedule_block:view
schedule_block:create
schedule_block:update
schedule_block:delete
recurring_schedule:view
recurring_schedule:create
recurring_schedule:update
recurring_schedule:delete
recurring_schedule_block:view
recurring_schedule_block:create
recurring_schedule_block:update
recurring_schedule_block:delete
schedule:view
resource_service:view
resource_service:create
resource_service:delete
service:view
service:create
service:update
service:delete
service:protected_metadata
bookable_slot:view
Create
Manage API Tokens
You may delete any of your existing tokens if they are no longer needed.
My first token

HAPIO

Your first request

Now that you have registered an account, created a project, and also created an API token for your project, you are ready to make your first request to the Hapio API. In order to authenticate your request, you must include your token in the **Authorization** header, using the bearer authorization scheme. For our first request, we will send a **GET** request to the endpoint /v1/project. This endpoint will simply return information about the project that the API token belongs to. This request can look like this:

```http
GET /v1/project HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
```

```php
use Hapio\Sdk\ApiClient;

$apiClient = new ApiClient('GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$project = $apiClient->projects()->getCurrentProject();
```

```javascript
import axios from 'axios';

const apiClient = axios.create({
    baseURL: 'https://eu-central-1.hapio.net/v1/',
    headers: {'Authorization': 'Bearer
GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});

apiClient.get('project').then(function (response) {
    const project = response.data;
});
```

It's important to note that in these examples, a dummy token will be included, since we want to show the entire request. You should of course always take the necessary steps to protect your tokens, and never show them to unauthorized people unless they are tokens intended to be public with limited abilities.

Here is the response:

HAPIO

```http
HTTP/2 200
date: Thu, 24 Aug 2023 08:39:02 GMT
content-type: application/json
content-length: 199
cache-control: no-cache, private
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
access-control-allow-origin: *
apigw-requestid: KKCN-gMUFiAEJZw=

{
    "id": "ac590ea9-e8a1-47eb-b301-452531ac962d",
    "name": "My first project",
    "enabled": true,
    "created_at": "2023-08-24T07:51:58+00:00",
    "updated_at": "2023-08-24T07:51:58+00:00"
}
```

Map your entities

You have now made your first request to the Hapio API, so let's get started with something a little more interesting. The first thing we need to do is to map our real-world entities to the entities that are available in Hapio. Let's say that we run a medical clinic, and want to use Hapio to manage appointments with doctors in this clinic. In this case, each doctor can be represented as a resource in Hapio, and each of the services that are offered can be represented as a service in Hapio. The clinic itself can be represented as a location in Hapio. Our mapping then looks like this:

Real world	Hapio
Doctor	Resource
Service	Service
Clinic	Location

This setup will allow us to easily add more doctors, services and clinics to our booking system as needed.

HAPIO

Create a resource

We will begin the process of implementing our real-world scenario in Hapio by creating a resource for one of our doctors, by sending a POST request to the endpoint /v1/resources:

```http
POST /v1/resources HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
Content-Type: application/json
Content-Length: 84

{
    "name": "Dr. Smith",
    "max_simultaneous_bookings": 1,
    "enabled": true
}
```

HTTP

```php
use Hapio\Sdk\ApiClient;
use Hapio\Sdk\Models\Resource;

$apiClient = new ApiClient('GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$resource = new Resource();
$resource->name = 'Dr. Smith';
$resource->maxSimultaneousBookings = 1;
$resource->enabled = true;

$resource = $apiClient->resources()->store($resource);
```

PHP

HAPIO

```JavaScript
import axios from 'axios';

const apiClient = axios.create({
    baseURL: 'https://eu-central-1.hapio.net/v1/',
    headers: {'Authorization': 'Bearer
GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});

apiClient.post('resources', {
    name: 'Dr. Smith',
    max_simultaneous_bookings: 1,
    enabled: true
}).then(function (response) {
    const resource = response.data;
});
```

The response for this request will include all information about our created resource:

```HTTP
HTTP/2 201
date: Thu, 24 Aug 2023 08:58:05 GMT
content-type: application/json
content-length: 282
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
access-control-allow-origin: *
cache-control: no-cache, private
apigw-requestid: KKFAihv0liAEJTw=

{
    "id": "c8924e50-af0d-47ae-bf5d-e184b1b59a60",
    "name": "Dr. Smith",
    "max_simultaneous_bookings": 1,
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "updated_at": "2023-08-24T08:58:04+00:00",
    "created_at": "2023-08-24T08:58:04+00:00"
}
```

HAPIO

Create a service

Now, it is time to create our first service by sending a **POST** request to the endpoint /v1/services:

```http
POST /v1/services HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
Content-Type: application/json
Content-Length: 297

{
    "name": "Physical Exam",
    "price": "149.000",
    "type": "fixed",
    "duration": "PT50M",
    "bookable_interval": "PT1H",
    "buffer_time_after": "PT10M",
    "booking_window_start": "PT2H",
    "booking_window_end": "P14D",
    "cancelation_threshold": "PT12H",
    "enabled": true
}
```

```php
use Hapio\Sdk\ApiClient;
use Hapio\Sdk\Models\Service;

$apiClient = new ApiClient('GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$service = new Service();
$service->name = 'Physical Exam';
$service->price = '149.000';
$service->type = 'fixed';
$service->duration = 'PT50M';
$service->bookableInterval = 'PT1H';
$service->bufferTimeAfter = 'PT10M';
$service->bookingWindowStart = 'PT2H';
$service->bookingWindowEnd = 'P14D';
$service->cancelationThreshold = 'PT12H';
$service->enabled = true;

$service = $apiClient->services()->store($service);
```

```JavaScript
import axios from 'axios';

const apiClient = axios.create({
    baseURL: 'https://eu-central-1.hapio.net/v1/',
    headers: {'Authorization': 'Bearer
GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});

apiClient.post('services', {
    name: 'Physical Exam',
    price: '149.000',
    type: 'fixed',
    duration: 'PT50M',
    bookable_interval: 'PT1H',
    buffer_time_after: 'PT10M',
    booking_window_start: 'PT2H',
    booking_window_end: 'P14D',
    cancelation_threshold: 'PT12H',
    enabled: true
}).then(function (response) {
    const service = response.data;
});
```

Again, the response for this request will include all information about our created service:

```HTTP
HTTP/2 201
date: Thu, 24 Aug 2023 09:07:53 GMT
content-type: application/json
content-length: 529
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
access-control-allow-origin: *
cache-control: no-cache, private
apigw-requestid: KKGchgzOliAEPQg=
```

```
{
    "id": "a09c4585-0519-44f6-a5ab-647aeffc6281",
    "name": "Physical Exam",
    "price": "149.000",
    "type": "fixed",
    "duration": "PT50M",
    "bookable_interval": "PT1H",
    "buffer_time_before": "PT0S",
    "buffer_time_after": "PT10M",
    "booking_window_start": "PT2H",
    "booking_window_end": "P14D",
    "cancelation_threshold": "PT12H",
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "updated_at": "2023-08-24T09:07:53+00:00",
    "created_at": "2023-08-24T09:07:53+00:00"
}
```


Create a location

Next up, we will create a location for our clinic by sending a **POST** request to the endpoint /v1/locations:

```
POST /v1/locations HTTP/2                                    HTTP
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer
GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
Content-Type: application/json
Content-Length: 149

{
    "name": "Perfect Health - Stockholm",
    "time_zone": "Europe/Stockholm",
    "resource_selection_strategy": "equalize",
    "enabled": true
}
```

```php
use Hapio\Sdk\ApiClient;
use Hapio\Sdk\Models\Location;

$apiClient = new ApiClient('GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$location = new Location();
$location->name = 'Perfect Health - Stockholm';
$location->timeZone = 'Europe/Stockholm';
$location->resourceSelectionStrategy = 'equalize';
$location->enabled = true;

$location = $apiClient->locations()->store($location);
```

```javascript
import axios from 'axios';

const apiClient = axios.create({
    baseURL: 'https://eu-central-1.hapio.net/v1/',
    headers: {'Authorization': 'Bearer
GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});

apiClient.post('locations', {
    name: 'Perfect Health - Stockholm',
    time_zone: 'Europe/Stockholm',
    resource_selection_strategy: 'equalize',
    enabled: true
}).then(function (response) {
    const location = response.data;
});
```

As with the previous requests, the response will include all information about our created location:

```http
HTTP/2 201
date: Thu, 24 Aug 2023 09:42:13 GMT
content-type: application/json
content-length: 347
cache-control: no-cache, private
```

```
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
access-control-allow-origin: *
apigw-requestid: KKLeXiW5FiAEMiA=

{
    "id": "2b113cf6-5d6c-4a31-bd21-88ba7fb440ea",
    "name": "Perfect Health - Stockholm",
    "time_zone": "Europe/Stockholm",
    "resource_selection_strategy": "equalize",
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "updated_at": "2023-08-24T09:42:13+00:00",
    "created_at": "2023-08-24T09:42:13+00:00"
}
```

Connect the resource with the service

Now that both our first resource and service are created, we need to associate them with each other. This tells Hapio that the resource (our doctor) is able to perform the service. We do this by sending a **PUT** request to the endpoint /v1/services/{service ID}/resources/{resource ID}:

```HTTP
PUT /v1/services/a09c4585-0519-44f6-a5ab-647aeffc6281
/resources/c8924e50-af0d-47ae-bf5d-e184b1b59a60 HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
```

```PHP
use Hapio\Sdk\ApiClient;

$apiClient = new ApiClient('GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$association = $apiClient->services()->associateResource(
    $service->id,
    $resource->id
);
```